Paiflow JRU Hackathon Event Primer

Updated Oct 12, 2026

Build an app that solves a real problem with automated payments through Paiflow. Choose your idea, build and integrate before the event; use October 14 to finish your app and demo.

01 At a glance

Item	Details
When and where	October 14, 2026 · José Rizal University · 10:30 AM–6:00 PM PHT
Format	One-day, in-person hackathon for students
Teams	10 teams of 1–4 people
Online onboarding	October 12, 2026 · 9:00 PM PHT
Registration closes	October 11, 2026 · 12:59 PM PHT
Event development time	Four hours, with a one-hour lunch break
Prize pool	PHP 10,000
Network	Stellar testnet

02 Event program

Online onboarding

> **October 12, 2026, at 9:00 PM PHT.** Hear the rules and watch a live Paiflow demo: build, deploy, scan the QR and see the payout. The meeting link will be provided through **blockhub.academy** or the student organisation representatives.

Build your app and integrate Paiflow before the event once team credentials are available. Ask organisers for credentials in advance. Onboarding covers setup and questions; October 14 is for finishing your app, verifying payouts and preparing the demo.

All times below are in Philippine Time (PHT).

02 Hackathon schedule

Time	Activity	What happens
10:30 AM–12:30 PM	**Development, part 1**	Finish integration and verify a confirmed payout
12:30–1:30 PM	**Lunch and checkpoint**	Optional coding; check your deployment and payout evidence
1:30–3:30 PM	**Development, part 2**	Complete integration and prepare your demo
3:00–3:30 PM	**Submission**	During the final 30 minutes of development, submit before 3:30 PM with your deployment ID and payout transaction
3:30–4:30 PM	**Pitching**	10 teams × 8 minutes (5-minute pitch and demo + 3-minute Q&A); random order drawn at 3:30 PM
4:30–5:00 PM	**Judging**	Judges deliberate and finalise scores
5:00–5:30 PM	**Announcement of winners**	Announce prize winners
5:30–6:00 PM	**Closing**	Closing remarks

03 Preparation and integration timeline

Starting now: choose a problem, talk to potential users and sketch your app. Register your team by October 11 at 12:59 PM PHT.

Before the event: build your app and integrate Paiflow. Boilerplate, open-source libraries, the starter and AI tools are allowed. Select team mode when your deployment and token are ready.

Wallet and payment preparation: prepare testnet wallets, add the exact USDC trustline if needed, and obtain funds. Flow deployment, Paiflow API integration and test payments are allowed before the event.

October 14, 10:30 AM PHT: finish your app and integration, verify the payout and prepare your demo. Use your team's deployment for qualifying evidence; shared-demo payments do not qualify.

Event development session: 10:30 AM–3:30 PM PHT, with lunch from 12:30–1:30 PM. Early integration is allowed. Lunch coding is optional. Submit by **3:30 PM PHT**.

04 The challenge

> **Dev mode will stay disabled for the duration of the hackathon.** Configure recipients and required values before deploying. Per-payment recipients, **Fill value via API** and **Dev values** are unavailable; standard Pay and Split flows remain available. `PAIFLOW_MODE=team` and `pnpm dev` do not enable flow Dev mode.

The theme is open. Start with a problem you or people around you have, then choose the Paiflow blocks that fit. These examples show how blocks combine and can inspire your own idea.

Paiflow deploys a flow you draw as Stellar contracts. Your app decides *who* pays and *why*; your flow decides *when* money moves and *where* it goes.

Idea	What it does	Paiflow pieces
Barkada bill splitter	A friend pays their share of a trip. The money automatically goes to friends who paid the group's expenses	`on_receive` + `split`, QR trigger page
Org dues and event tickets	QR payments for dues or tickets split between the org treasury and venue	`on_receive` + `split`, embed + live feed
Creator collab tip jar	Shared-project tips split between collaborators at agreed percentages	`on_receive` + `split`
Sari-sari store checkout	A customer scans and pays; the supplier's share is sent immediately	`on_receive` + `split` or `pay`
Digital baon	A parent sets up scheduled weekly allowance payments	`on_schedule` + `pay`
Tutor or club subscription	One configured member approves an allowance for recurring charges	`subscription` + allowance page
Freelance deadline escrow	Client pays up front; acceptance before the deadline pays the freelancer, otherwise the client is refunded	`on_receive` + `condition` (`time_before`) + two `pay` branches
Rewards from a Web2 app	A pre-funded pool pays a configured recipient when the app reports a completed goal	`web2_webhook` + authenticated callback + `pay` or `split`
Transparent donation drive	Donations split among beneficiaries, with a public feed showing payouts	`on_receive` + `split`, embed + live feed

Strong projects have a real user, a flow that fits their problem, and a demo with visible payouts.

For technical setup and example payment flows, see the Paiflow Guide, API quickstart and AI context pack. Follow this primer and the organisers' instructions for preparation and integration timing. Confirm feature availability with support before choosing an idea.

> **Support contact:** paiflow+jru.hack@artisam.xyz.

Testnet funds have no real-money value. Never store wallet secret keys in your app.

Set up Paiflow once for the business, group or branch your app serves. For example, Campus Snacks represents one branch of a snack shop. Each customer pays separately, but every payment uses the same setup: 90% goes to the shop and 10% to the student organisation. You don't need a new setup for each customer.

If your idea requires a separate payment setup for every customer, explain that as a future feature in your pitch.

05 Prizes

Qualification: a confirmed team deployment and a verifiable on-chain payout from it. Submit your deployment ID and payout transaction through the submission form by **3:30 PM PHT on October 14**. Non-qualifying teams may pitch but cannot rank or win prizes.

Scoring: Creativity 30%, Integration 40%, Real-world usefulness 30%; averaged across judges. Integration rewards fit over block count. Works end to end allows up to 10 points for a live demo or 7 for a recording of your actual app, deployment and verifiable payout.

Pitch: five minutes for pitch and demo, then three minutes for Q&A; eight minutes total per team, with a strict timer. Order is drawn randomly at 3:30 PM. Set up during the previous team's Q&A. A recording may replace a failed live demo; the 7-point limit then applies.

PRIZE POOL

PHP 10,000

For qualifying teams only. Prizes go by QR Ph to the member named in your submission.

Winning projects are featured on the Paiflow homepage and socials.

06 Event support and coordination

Registration and resources

Register by **October 11, 12:59 PM PHT**. Solo teams are welcome; two or more members are encouraged. Organisers will coordinate with the student organisation if more than 10 teams apply.

Use the starter, Campus Snacks sample and its live app. For wallet setup, flow deployment and starter configuration, follow the Paiflow Guide. Follow this primer for JRU event rules and judging. Resources may change without prior notice.

What to prepare and bring

Install **Node.js 22, pnpm 10.4.1 using npm, Git and Freighter**. Follow the Paiflow Guide linked above for setup and version checks. Have a GitHub account. Run `pnpm install` in the starter to install its app dependencies. Prepare funded testnet wallets; USDC needs its exact trustline and separate funding. Friendbot supplies XLM only.

Bring your own internet connection, such as mobile data or a hotspot. Wallet and funding support will be available on site, with fallback wallets as a last resort. Organisers retain their keys; never use them for real funds or import them into a wallet used for real money.

Shared rules: use Paiflow for the core money flow. Testnet only; no real funds. Keep API tokens on your server and wallet secrets out of your app. No API or relayer load-testing or spam; rate limits apply. Organisers decide disputes. The beta app collects PostHog usage analytics.